AGENTIC AI LANGGRAPH

→ Agentic AI

It is an AI system designed to take actions & make decisions autonomously to achieve a goal.

Instead of simply generating an answer, the system can:

- Reason about a task
- Decide what to do next
- Call tools
- Use API's
- Read/write data
- Observe results
- Re-evaluate
- Retry
- continue until the objective is achieved

↳ Basic Architecture

User Goal
↓
Agent / LLM
↓
Decide Next step
↓
Tool / API
↓
Result
↓
Re-evaluate
↓
Decide Again
↓
Final output

↳ Execution Loop

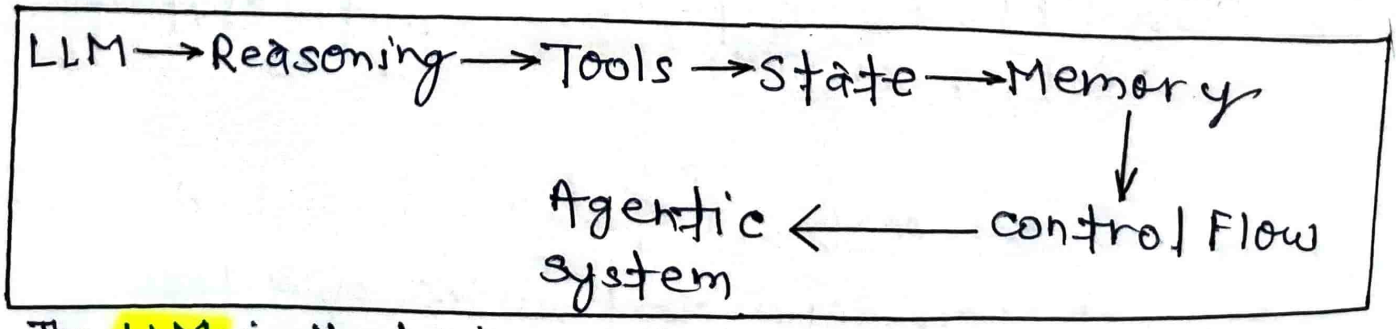
Reason
↓
Act
↓
observe
↓
Reason
↓
Act
↓
observe
↓
.....

→ Generative AI Vs Agentic AI

Feature	Generative AI	Agentic AI
Main Purpose	Generate content	Achieve a goal
Behaviour	Reactive	Proactive
Execution	Usually single-step	Multi-step
Tool Usage	Limited/none	Tools & API's
Decision Making	User-driven	AI driven
Loops	Usually absent	core component
State	Usually stateless	stateful
Control Flow	Simple	Dynamic
Memory	Limited / context-based	state + persistance possible
Example	Generate an email	Research → verify → write report

Note:- Agentic AI doesn't replace "Generative AI".

Instead:



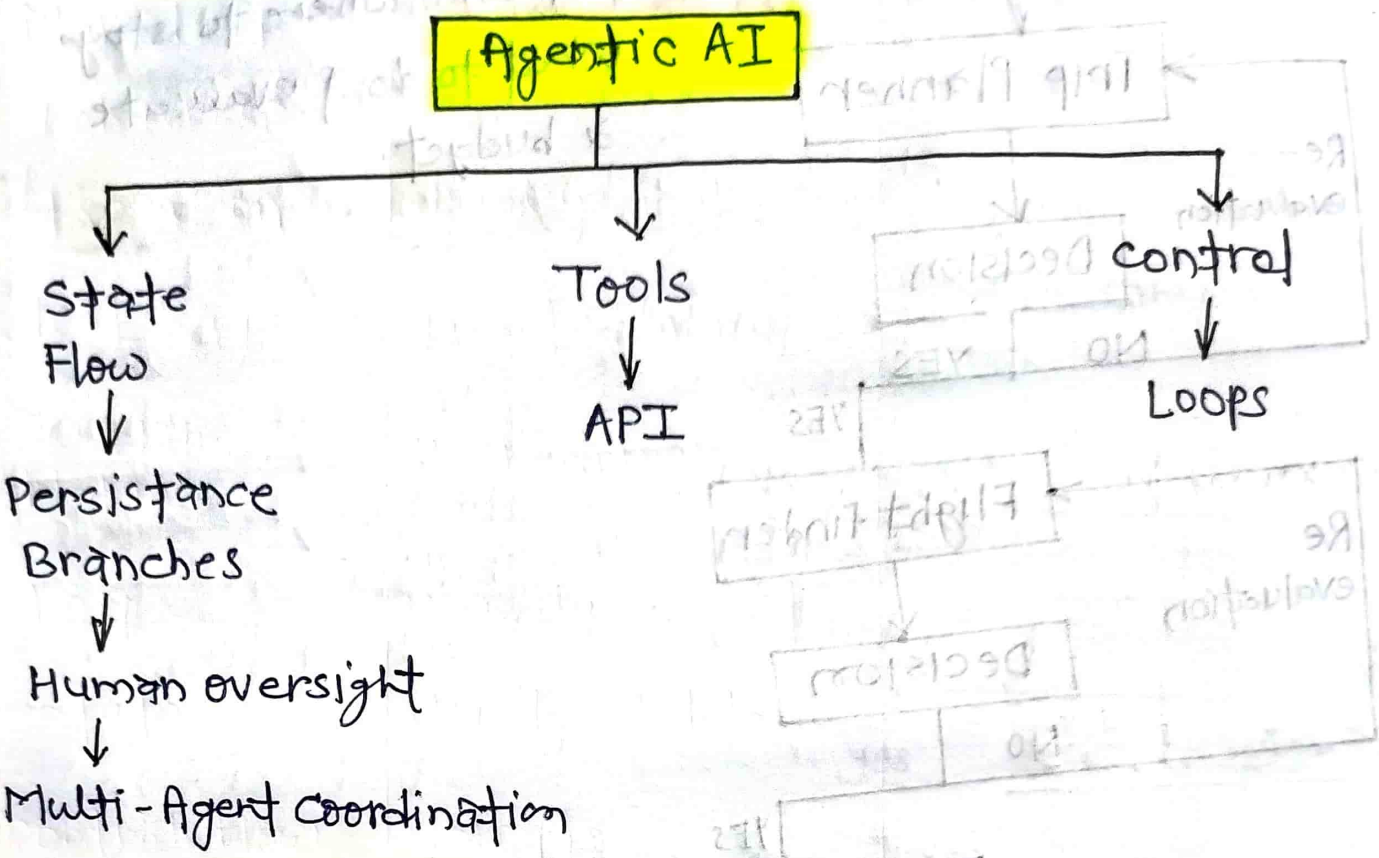
The LLM is the brain, while it surrounding architecture enables it to behave like an agent.

→ The Agent Spectrum

LEVEL	What it does	Example
Level: 0 Pure generation	LLM answer a prompt, no tools	ChatGPT writing a poem
Level: 1 Tool using LLM	LLM calls a tool once, return answer	LLM checking weather via a tool
Level: 2 ReAct Agent (We are here)	LLM LOOPS: think → call tool ↓ call tool ← think ← observe, until done	Agent answering "What's the weather and convert it to Farhrenheit" using two tools
Level: 3 Stateful Agent	Agent remembers across turns, has persistent memory, can pause for humans	Customer support bot that remembers you past tickets
Level: 4 Multi-agent	Mutiple agents coordinate to achieve a goal, each with their own role	Research: Planner + researcher agent + writer agent
Level: 5 Autonomous Agent	Long-running, self-correcting, self-improving, plans across day/weeks	Devin like coding agent that ships PRs

→ What make a system Truly Agentic

↳ Important components:



↳ key components:

(1) state : It remembers what happened

(2) Persistance : It can be paused & resumed

(3) control Flow: It can branch, loop & coordinate

(4) Tools at scale: not just 2-3 tools, sometimes hundreds

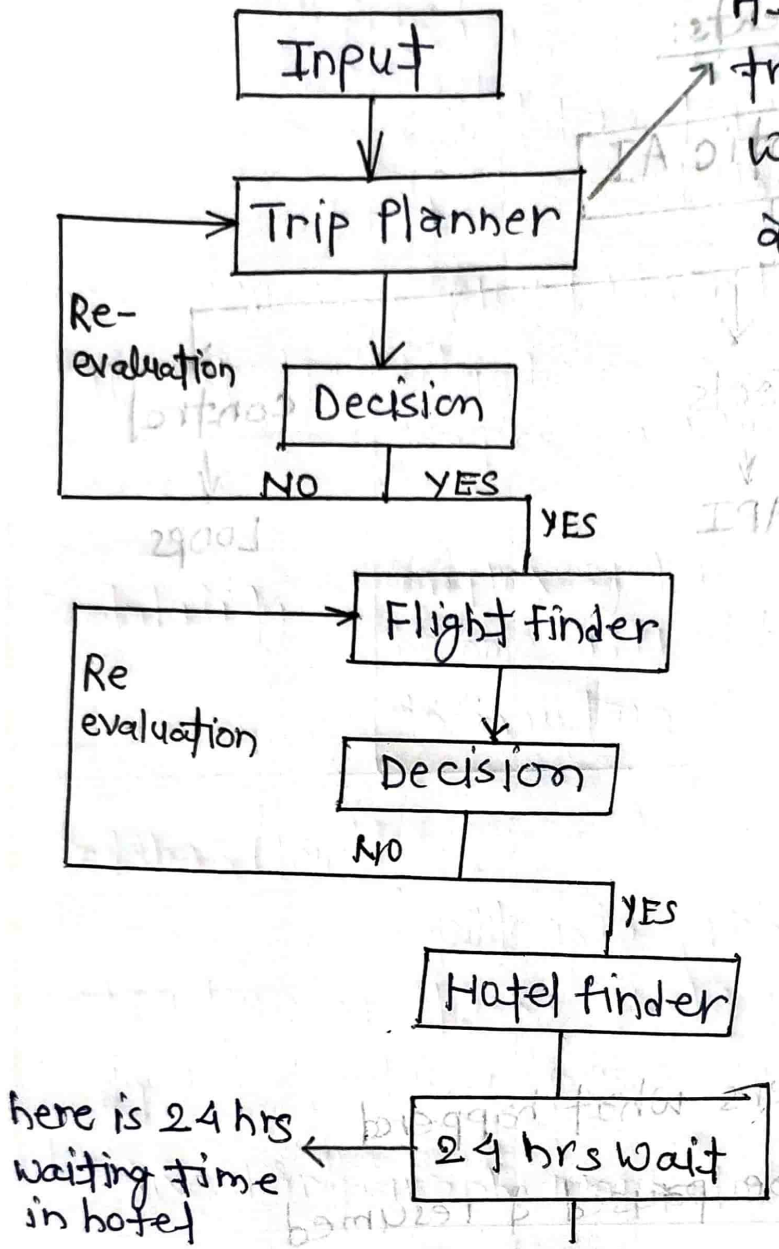
(5) Human oversight: humans can approve sensitive actions

(6) Multi-agent coordination: multiple specialized agents working together.

Note: - In Langchain "create_agent()", "agent" means LLM that can use tools in loops.

Thats why it is not agentic

→ Why Lang-Graph



↳ core idea: "Langchain" helps us work with LLM's & components;

"Lang Graph" helps us to orchestrate stateful, controllable AI workflows It represents an AI workflow as a graph.

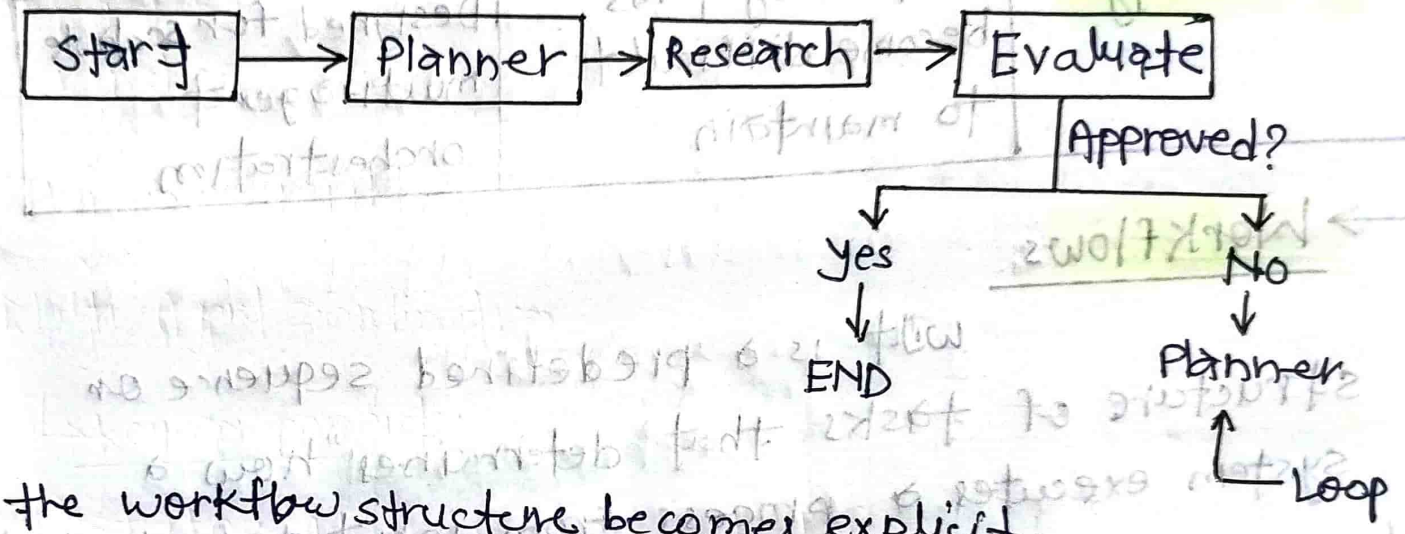
- The graph consists primarily of

State + Nodes + Edges

These three are the core DNA of Langraph:

↳ Langgraph Main Advantage

LangGraph models complex execution explicitly as a Graph



the workflow structure becomes explicit.

↳ Langchain Vs LangGraph

Problem	Langchain	LangGraph
Loops & Re-evaluation	Manual "while" loop & cond^n handling make workflows messey	Native graph look with conditional edges make workflows clean & scalable
Multi-Agent State Management	shared memory between agents must be managed manually	centralized shared state automatically flows across all agents.

Problem	Langchain	Langgraph
Long waiting Tasks	Requires external databases or schedulers to pause/ resume workflows	Build-in persistent & resumble workflows handle waiting naturally
Human-in-the-loop Approval	Approval & branching logic become complex quickly	Human decisions are handled clearly using graph transitions
Scalability	Complex systems become difficult to maintain	Designed for scalable multi-agent orchestration

→ **Workflow**

It is a predefined sequence or structure of tasks that determines "how a system executes a process from input to final output".

↳ **Five LangGraph workflow Types**

(1) **sequential workflow** :- Tasks run one by one
A → B → C

(2) **Parallel workflow** :- Tasks run simultaneously

(3) **Conditional workflow** :- choose a path based on a condition

(4) **Iterative workflow** :- Repeats until the goal is met

(5) **Human-in-the-loop (HITL)** - Pauses for human input/approval.

→ Langgraph core Architecture

State + Node + Edges
↓
stateGraph
↓
workflow

A graph defines what data exists, what work is performed, & how execution moves beth tasks.

→ **State:** stores data shared between nodes.

↳ **Ex**

input → "write a blog" is stored in state.

↳ **code**

```
class State (TypedDict):
    input: str
    output: str
```

↳ **Flow**

Node A → reads state → does work → updates state
↓
reads updated state ← Node B

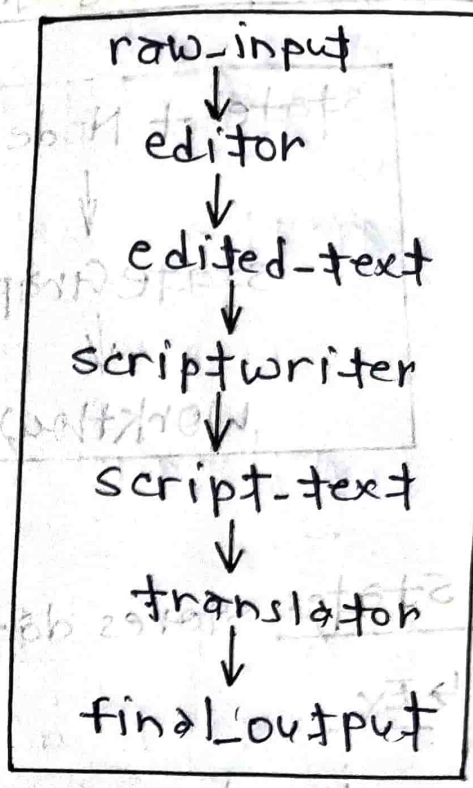
↳ **State with TypedDict:** "TypedDict" it is a python hint used to define the structure / schema of a dictionary, specifying expected keys & their value types

In short:- defines the expected structure of a dictionary.

• Ex

```
from typing import TypedDict
class PiplineState(TypedDict):
    raw_input : str
    edited_text : str
    script_text : str
    final_output : str
```

• Flow



→ **Nodes** : It is an individual unit of work in a LangGraph workflow.

Technically, it can be a Python function that:

(1.) Receives the current state

(2) Performs some operation.

(3) Returns state updates

↳ **Ex code**

```
def editor_node(state : PipelineState) -> dict:
    text = state["raw_input"]
    return {
        "edited_text" : text}
```

- Flow

state → editor_node() → edited_text

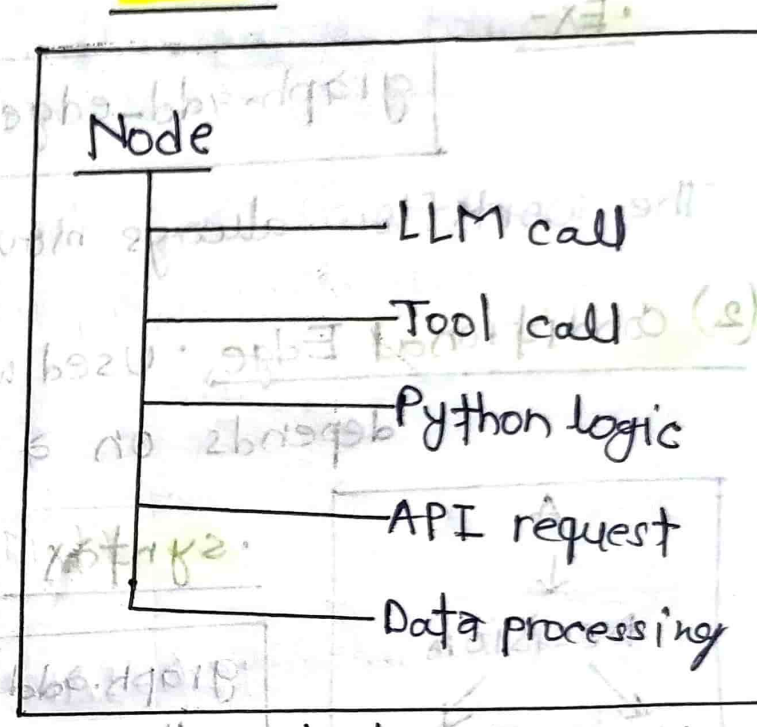
↳ Nodes can Perform Different tasks

It is isn't necessarily an LLM call:

It can perform:

- LLM invocation
- API calls
- Tool execution
- Data processing
- Validation
- Calculation
- Database operations
- Routing/decision logic

- Ex



That makes nodes the actual workers of the grap

→ Edges

It defines the connection & execution path beth nodes.

↳ Ex

```
graph.add_edge (START, "editor")
graph.add_edge ('editor', "scriptwriter")
graph.add_edge ('scriptwriter', "translator")
graph.add_edge ('translator' , END)
```

↳ Flow

START → Editor → scriptwriter → Translator → END

↳ Types of Edges

(1) Normal Edge : It is a fixed connection

• Ex-

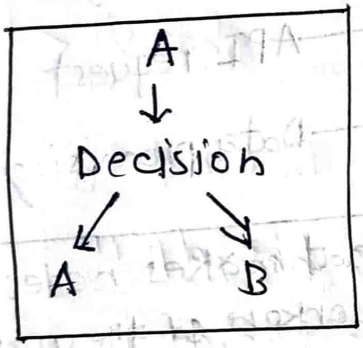
```
graph.add_edge("A","B")
```

• Flow

A → B

The workflow always moves from A to B.

(2) Conditional Edge : Used when the next step depends on a condition or decision.



• syntax

```
graph.add_conditional_edges(
  "source_node", routing_function,
  {"condition_1 : "next_node_1",
   "condition_2 : "next_node_2" })
```

In short:- Decision base routing between nodes.

→ START & END

LangGraph provides special marker START & END

↳ **START** - Represents the entry point of the graph

• Ex

```
graph.add_edge(START, "editor")
```

means:

START → editor

↳ END : Represents the completion of the workflow

• Ex

```
graph.add_edge("translator", END
```

• Means:

Translator → END

→ Sequential Workflow

Here the tasks execute one after another in a fixed, predefined order

In short: Node A → Node B → Node C → Node D

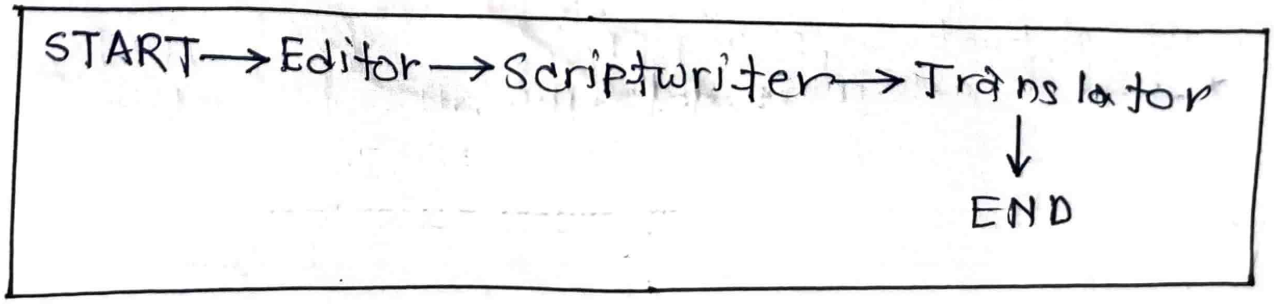
↳ Key Notes:

- Each Node runs after the previous node completes.
- The output of one node can become the input for the next.
- Execution path is predictable & linear.
- Uses normal add_edge() connections.
- starts from START & finishes at END.

↳ Ex code

```
graph.add_edge(START, "editor")
graph.add_edge("editor", "scriptwriter")
graph.add_edge("scriptwriter", "translator")
graph.add_edge("translator", END)
```

↳ Flow

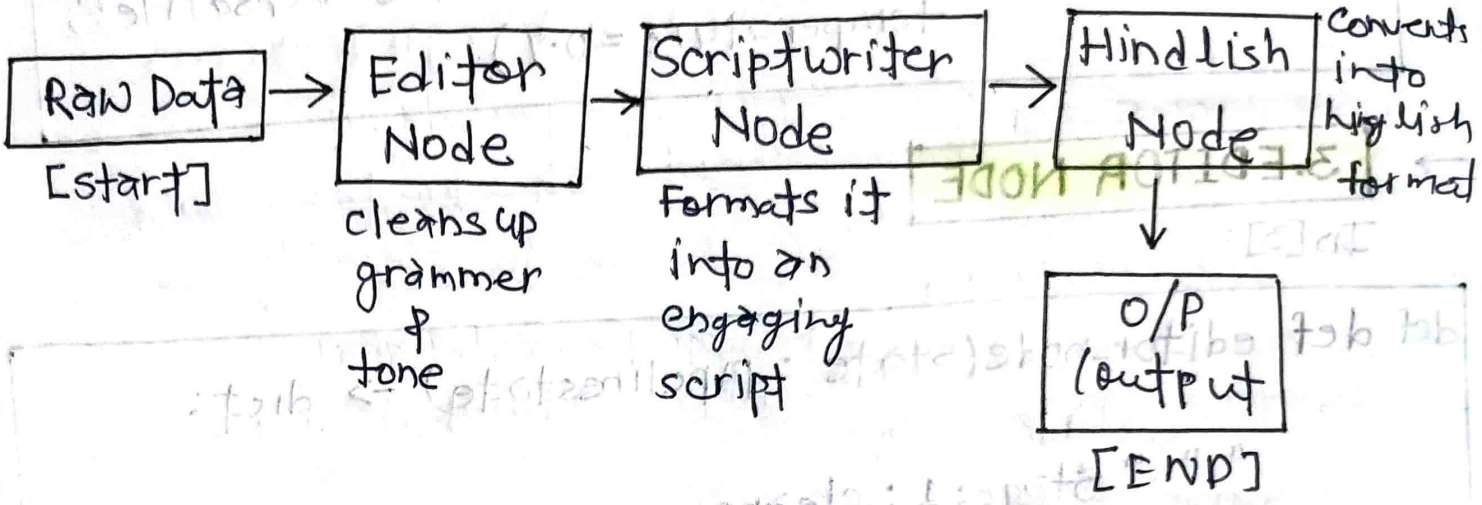


PROJECT : 1
SCRIPT-FLOW AI

(Using sequential workflow)

↳ overview

- Goal - Transform a raw script idea into a polished, engaging Hinglish script



- core concept : Demenstrate LangGraph sequential workflow, where each node executes in a fixed order.
- Tech stack : LangGraph , Langchain, Groq LLM & Python dotenv

→ 1. IMPORTS & STATE

In[1]:

```
import os
from typing import TypedDict

#creating state
class pipelinestate(TypedDict):
#Defines the pipeline state structure
    raw_input : str  #stores raw input
    edited_text : str  # stores edited text
    script_text : str  #stores generated script
    final_output : str #stores final output
```

2. LLM SETUP

In[2]:

```
from langchain_groq import ChatGroq
from dotenv import load_dotenv
load_dotenv()
llm = ChatGroq(model="llama-3.3-70b-versatile",
               temperature=0.7)
```

3. EDITOR NODE

In[3]:

```
def editor_node(state : pipelinestate) -> dict:
    #defines Editor node
    """ Stage 1 : cleans up grammer, removes
    typos, & refines the tone. """
    prompt = ( "you are a expert copy editor .clean
    up the following raw text."
                "Fix any grammatical errors,
    spelling mistakes, & smooth out the transition
    flow"
                "while keeping the core message
    intact. Return only the edited text.\n\n"
                f"Text: \n {state['raw_input']})
    #Gets raw input from source
    response =llm.invoke(prompt)

    return { "edited_text": response.content.strip()}
    #updates state with edited text
```

4. SCRIPTWRITER NODE

In[4]:

```
def scriptwriter_node(state:piplinestate) -> dict:
#Define scriptwriter node
    """stage 2 : Formats the clean text into an
    engaging videoscript style""""
        print("\n---[stage 2] Executing scriptwriter
        Node ---")
        prompt=("you are a charismatic youtube
        content creator. Take this edited text & transform"
        "it into a highly engaging, punchy,
        conversational video hook. Make it sound "
        "like a real person speaking passionately.
        Return only the script content. \n\n"
        f"Edited Text: \n{state['edited_text']}"
        #Gets edited text from state
        response = llm.invoke(prompt)
        return {"script_text: response.content.strip()}
```

5. TRANSLATOR NODE

In[5]:

```
def translator_node(state:pipelinestate) -> dict:
    """stage 3: Translates the script into natural
    flowing Hinglish."""
    print("\n---[stage 3] Executing Hinglish
    Translator Node ---")
```

```
prompt = ("you are an expert content localizer
for the Indian market. Take the following script".
          "and convert it into natural, flowing
'Hinglish'. Do not simply translate it
sentence-by-sentence"
          "or repeat information. Alternating
comfortably between Hindi & English phrases
just like"
          "an intelligent tech educator would
speak naturally on a live stream. Keep the
energy high!"
          "Return only the final Hinglish
text. \n\n"
          f"Script: \n{state['script_text']}")
response = llm.invoke(prompt)
return {"final_output": response.content.strip()}
```

6. GRAPH CREATION

In [6]:

```
from langgraph.graph import StateGraph,
START, END

graph = StateGraph(pipelineState)
#creates graph using our state
graph.add_node("editor", editor_node)
graph.add_node("scriptwriter", scriptwriter_node)
graph.add_node("translator", translator_node)
```

→ 7. SEQUENTIAL EDGES

In [7]:

```
graph.add_edge(START, "editor")
graph.add_edge('editor', "scriptwriter")
graph.add_edge('scriptwriter', "translator")
graph.add_edge('translator', END)
```

- Flow - START → Editor → scriptwriter → Translator → END

→ 8. COMPILE, EXECUTE & OUTPUT

In [8]:

```
app= graph.compile()
#Compile graph into executable application
result=app.invoke({
        "raw_input": "AI agents are ....."})
print("your result are :-\n\n")
print(result['final_output'])
```

* **StateGraph**: LangGraph class used to create & define a workflow graph by connecting state, Nodes & Edges

syntax:

```
graph=StateGraph(state)
```

→ Reducer

Defines how multiple updates to the same state key are combined.

- Prevents state overwrite.
- Useful in parallel workflows.
- Merges outputs from multiple nodes.

↳ Ex

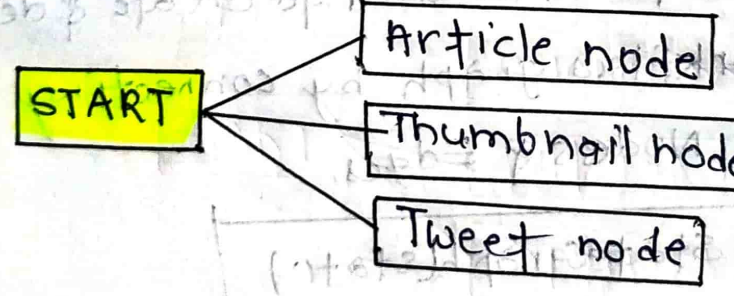
```
def merge_dict(existing, new):
    return (**existing, **new)

class State(TypedDict):
    scores: Annotated[dict, merge_dict]
```

→ Fan-Out & Fan-In

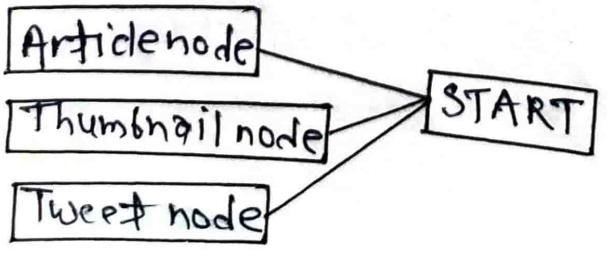
↳ **Fan-out (splitter)** : Multiple nodes running in parallel.

It splits workflow into parallel branches.



↳ **Fan-In (Merger)**: Multiple node → one node. The next next node wait for all branches to complete.

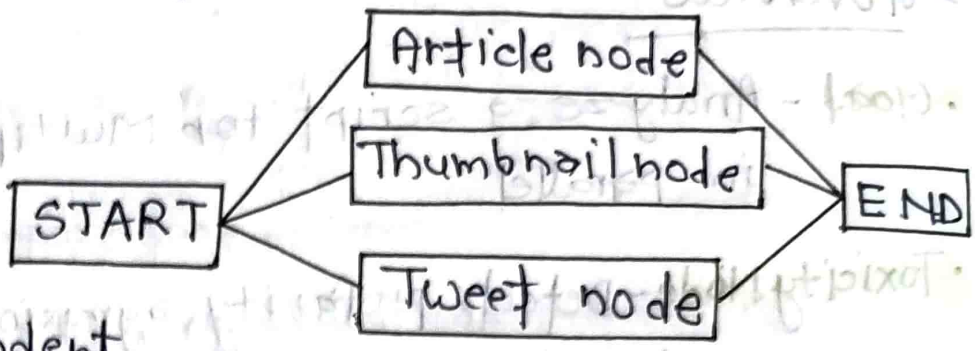
It Merges parallel branches after they complete



→ **Parallel workflow** here multiple node execute simultaneously instead of one after another.

↳ **Key points**

- Nodes run in parallel
- Useful for independent tasks
- Reduces overall execution time
- Multiple nodes can update the same state.
- Reducers contiine



↳ **Ex edge**

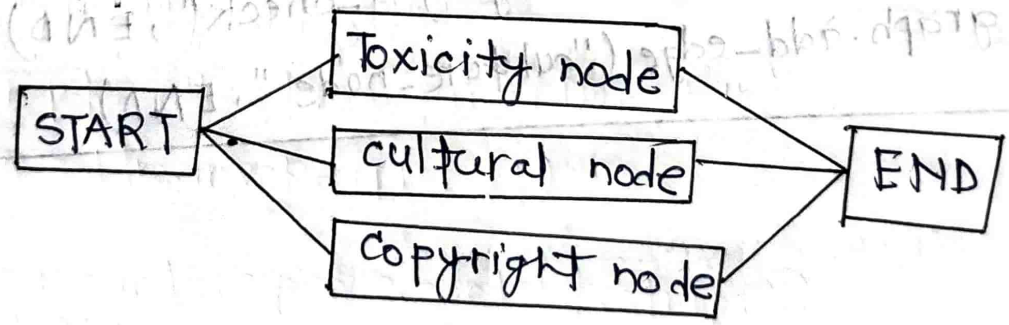
```
graph.add-edge (START, "toxicity_node")
graph.add_edge(START, "copyright_check")
graph.add_edge (START, "culture_node")
graph.add-edge("toxicity-node", END)
graph.add_edge ("copyright_check", END)
graph.add_edge ("culture_node ", END)
```

PROJECT : 2
AI CONTENT SAFETY ANALYZER

(Using Parallel workflow)

Overview

- **Goal** - Analyze a script for Multiple safety risks in parallel
- **Toxicity Node** - Detects profanity, agression, hate speech & toxicity.
- **Copyright Node** - checks plagiarism, originality & political sensitivities.
- **cultural Node** - Detects culutral, regional & political sensitives
- **Parallel workflow** - All 3 nodes analyze the same raw-text simultaneously
- **Reducer** : Merges all node results into the same safety-scores dictionary



1. IMPORTS

In[1]:

```
import os
from typing import TypedDict, Annotated
#Importing TypedDict for state structure & Annotated
  for reducers.
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langgraph.graph import StateGraph, START, END
```

2. ENVIROMENT & LLM SETUP

In[2]:

```
load_dotenv()
llm = ChatGroq(model="llama-3.3-70b-versatile",
               temperature=0.1)
```

3. CUSTOM REDUCER & STATE

In[3]

```
def merge_score_dicts(existing:dict, newupdate:dict)
  -> dict:
#Reducer Function to merge safety score dictionaries
    if existing is None:
      return newupdate
    return {**existing, **newupdate}
#state definition
class AnalyzerState(TypedDict):
    raw_ext:str
    safety_scores: Annotated[dict[str,int],
    merge_score_dicts]
    #safety scores with custom reducer.
```

→ 4. TOXICITY NODE

In[4]

```
def toxicity_node(state : AnalyzerState) -> dict:
#Node for checking toxicity & hate speech
    Print ("\n [Branch 1] Analyzing Toxicity &
           Hate Speech..")
    prompt = (" Enter required Prompt")
    response = llm.invoke(prompt)
    try: score = int (response.content.strip())
    #Try converting LLM response to integer
    except ValueError:
        score=0
    #Handle invalid/non numeric response
     return {"safety_score" : {"toxicity_level":score}}
```

→ 5. COPYRIGHT NODE

In[5]:

```
def copyright_node(state :AnalyzerState) -> dict:
    print ("\n [Branch 2] Analyzeing Copyright &
     originality Risks...")
    prompt = (" Enter Required Prompt")
response = llm.invoke(prompt)
try:
    score = int(response.content.strip())
except ValueError:
    score =0
return {"safety_scores" : {"copyright_risk": score}}
```

6. CULTURAL NODE

In [6]:

```
def culture_node(state: AnalyzerState) -> dict:
    print("\n[Branch 3] Analyzing Regional &
    cultural sensitivity...")
    prompt = ("Enter required prompt")
    response = llm.invoke(prompt)
    try:
        score = int(response.content.strip())
    except ValueError:
        score = 0
    return {"safety_scores": {"cultural_insensitivity":
     score}}
```

7. GRAPH CREATION, NODES & EDGES

In [7]:

```
buider = stateGraph(AnalyzerState)
#create the Langgraph workflow
builder.add_node("toxicity_node", toxicity_node)
builder.add_node("copyright_node", copyright_node)
builder.add_node("culture_node", culture_node)
builder.add_edge(START, "toxicity_node")
builder.add_edge(START, "copyright_check")
builder.add_edge(START, "culture_node")
builder.add_edge("toxicity_node", END)
builder.add_edge("copyright_check", END)
builder.add_edge("culture_node", END)
```

8. COMPILE, INPUT, EXECUTE & OUTPUT

In[8]:

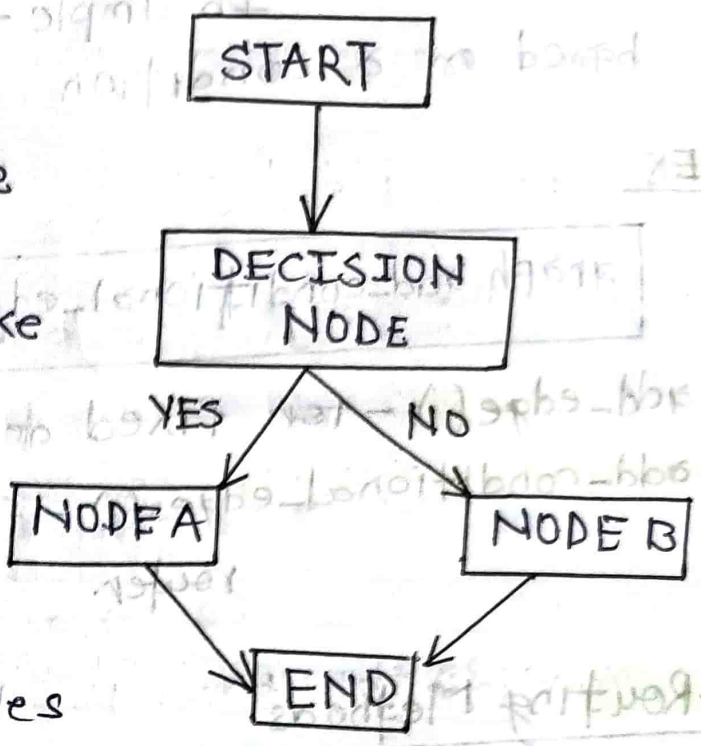
```
app=builder.compile()
sample_script=""" Enter any script """
intial_state={
#create intial state
      "raw_text": sample_script
       #Input text
       "safety_score": {}
       #start with empty safety scores
         }
final_state=app.invoke(intial_state)
#Execute the LangGraph workflow
print(final_state["safety_scores"])
```

→ Conditional Workflow

Where the next node is selected dynamically based on the current state.

↳ Key points

- Uses conditions to decide the next step.
- The flow is not fixed like a sequential workflow
- A Router Function manually make the decision.
- can route different nodes depending on the result.
- Useful for classification, decision making & multi-route systems.
- Routing can be rule based (if/else) or LLM based.



↳ Two Major things -

(1.) Router Function : It is a normal Python function that reads the state & decides which node should execute next.

- Ex code

```
def router(state):
    if state["type"] == "academic"
        return "academic_node"
    else:
        return "general_node"
```

- Router - Make the routing decision, not actual task.

(2) add_conditional_edges()

Used to create dynamic connections between nodes.

In simple - It chooses the next node based on a condition or router function.

- Ex

```
graph.add_conditional_edges("classifier", router)
```

- add_edge() - for fixed destination
- add_conditional_edges() - for destination decided by router

→ Routing Methods

- Routing - It decides which node should execute based on current state or condition.

(1) Rule-Based Routing

Uses python conditions (if/elif/else) to decide the next node.

- ex

```
def router(state):
    if state["type"] == "fee":
        return "fee_node"
    elif state["type"] == "academic":
        return "academic_node"
    else:
        return "general_node"
```

- Flow

```
state → Python conditions → Next Node
```

(2) LLM-Based Routing

Uses an LLM to analyze the query & decide its type, then routes accordingly

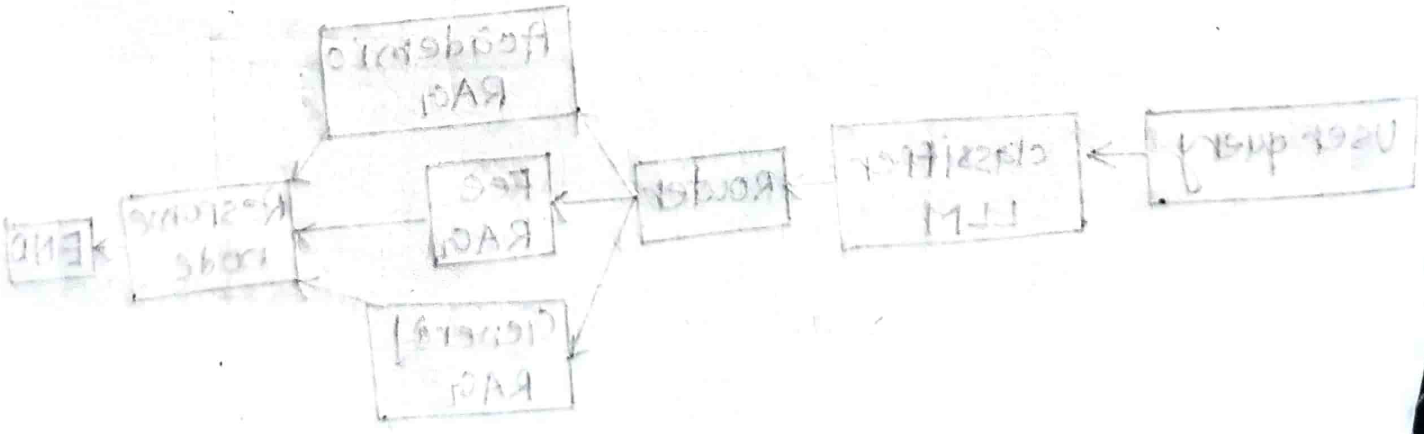
- Ex

```
def router(state):
    result = llm.invoke(
        f"Classify: {state['query']} as
fee, academic, or general")
    return
result.content.strip().lower() + "_node"
```

Flow: Query → LLM → Classification → Node

↳ Difference beth RuleBased & LLM Based

Rule Based	LLM Based
(1) Uses Python condns	(1) Uses LLM
(2) Fast	(2) slower
(3) Predictable	(3) More flexible
(4) Best for clear rules	(4) Better for ambiguous queries.



PROJECT-3
SMART COLLEGE ASSISTANT

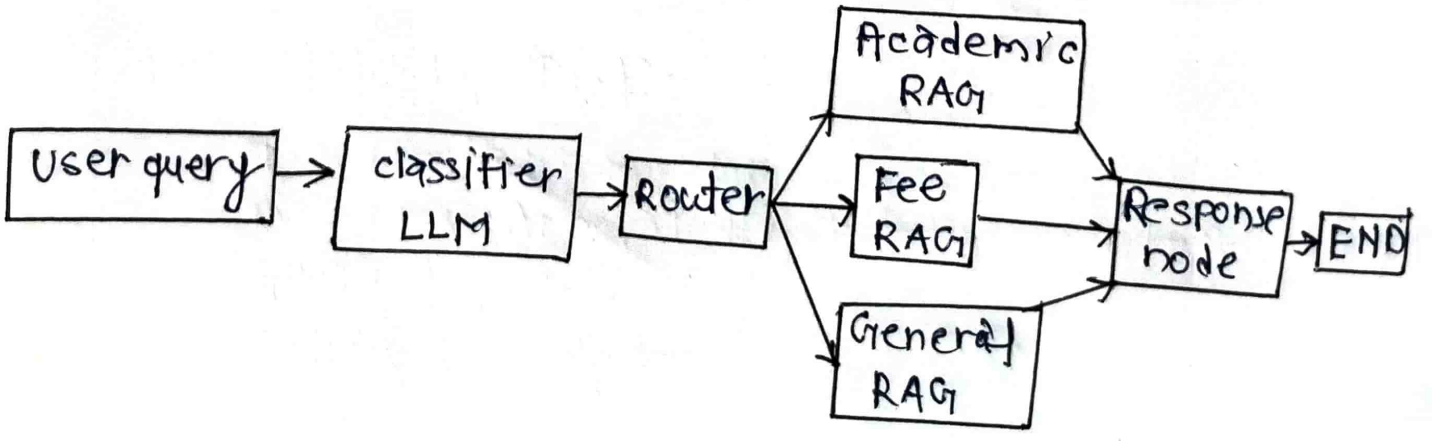
Conditional WorkFlow + RAG

→ **Overview**

It is AI powered college chatbot that answers academic, fee & general student queries by dynamically routing each query to the appropriate processing path.

- **LLM classifier** - identifies query type
- **Router** - selects the correct path
- **Academic RAG** - searches the academic handbook
- **Fee RAG** - searches the fee structure
- **General Node** - answers without retrieval
- **Response Node** - generates the final personalized answer
- **Uses FAISS + HuggingFace Embeddings** - for document retrieval.
- Uses add_messages reducer for conversion history

↳ **Flow**



→ **1. IMPORTS**

In[1]:

```
import os
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
#Reducers for Message history
from langgraph.graph import StateGraph,
START, END
from langchain_groq import ChatGroq
from langchain_community.document_loaders import
PyPDFLoader
from langchain_text_splitters import
RecursiveCharacterTextSplitter
#split documents
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import
FAISS
#FAISS is a Vector database
from dotenv import load_dotenv
#Load .env variables
load_dotenv()
#Load API Keys from .env
```

* **FAISS - (Facebook AI similarity search)** : Its a vector search library used to store embeddings & quickly find similiar/relevant documents.

- **Main Advantage**: very fast similarity search, especially for large amount of vector data.

2. RAG RETRIVERS & STATE

In [2]:

```
embeddings = HuggingFaceEmbeddings(model_name=
    "sentence-transformers/all-MiniLM-L6-v2")

def build_retriver(pdf_path:str):
    loader = PyPDFLoader(pdf_path)
    document = loader.load()          #Load PDF document

    splitter = RecursiveCharacterTextSplitter(
            chunk_size=800, chunk_overlap=100)
    vectorstore = FAISS.from_documents(chunks,
            embeddings)
    #store chunks as vectors in FAISS
    return vectorstore.as_retriever(search_kwargs=
        {"k": 4})

academic_retriever = build_retriver("academics-
                    handbook.pdf")
fee_retriever = build_retriver("fee-structure.pdf")
#created seperate retrievers for different knowledge
  sources.

llm = ChatGroq(model="llama-3.3-70b-versatile",
      temperature=0.4)
#created the LLM used by classifier & response nodes
======
#STATE
======
class State(TypedDict):
    programme: str
    messages: Annotated[list, add_messages] #chat history.
    query_type: str
    retrieved_context: str
```

→ 3. CLASSIFIER NODE & ACADEMIC RAG NODE

In[3]:

```
def classifier_node(state: State) -> dict:
    last_message = state["messages"][-1].content
    prompt = ("Enter prompt for the LLM to classify
              the query - ")
response = llm.invoke(prompt)
category = response.content.strip().lower()
if "academic" in category:
    category = "academic"
elif "fee" in category:
    category = "fee"
else:
    category = "general"

return {"query_type": category}
#update state with query category

==========
#ACADEMIC RAG NODE
==========
def academic_rag_node(state: State) -> dict:
    query = state["messages"][-1].content
    #Get the user's latest query
    docs = academic_retriever.invoke(query)
    #search the academic knowledge base
    context = "\n\n".join([doc.page_content for
              doc in docs])
    return {"retrieved_context": context}
```

4. FEE RAG NODE, GENERAL NODE & RESPONSE NODE

In[4]:

```
# FEE RAG NODE
def fee_rag_node(state:State) -> dict:
    query = state["messages"][-1].content
    docs = fee_retriever.invoke(query)
    context = "\n\n".join([doc.page_content for
        doc in docs])
    return {"retrieved_context": context}

# GENERAL NODE
def general_node(state:State) -> state:
    return {"reterieved_context": "NO_RETRIEVAL_NEEDED"}

# RESPONSE NODE
def response_node(state:State) -> dict:
    query = state["messages"][-1].content
    programme = state.get("programme", "Unknown")
    context = state["retrieved_context"]
    if context == "NO_RETRIEVAL_NEEDED":
        prompt = ("Enter a response prompt to
            handle general queries without RAG")
    else:
        prompt = ("enter prompt to handle fee
            queries using retrieved context")
response = llm.invoke(prompt)
return {"messages": [("ai", response.content.strip())]}
```

5. ROUTER FUNCTION

In [6]:

```
def route_query(state:State):
    if state["query_type"] == "academic"
        return "academic_rag"
    elif state["query_type"] == "fee"
        return "fee_rag"
    else:
        return "general"
```

6. NODES & EDGES

In [6]:

```
graph = StateGraph(state)

graph.add_node("classifier",classifier_node)
graph.add_node("academic_rag",academic_rag_node)
graph.add_node("fee_rag", fee_rag_node)
graph.add_node("general",general_node)
graph.add_node("response", response_node)

graph.add_edge(START, "classifier")
graph.add_conditional_edges("classifier", route_query)
# Dynamically choose the next node using using router
graph.add_edge("academic_rag","response")
graph.add_edge("fee_rag","response")
graph.add_edge("general","response")
graph.add_edge("response",END)
app=graph.compile()
```

6. USER PROGRAMME SELECTION & CHAT LOOP

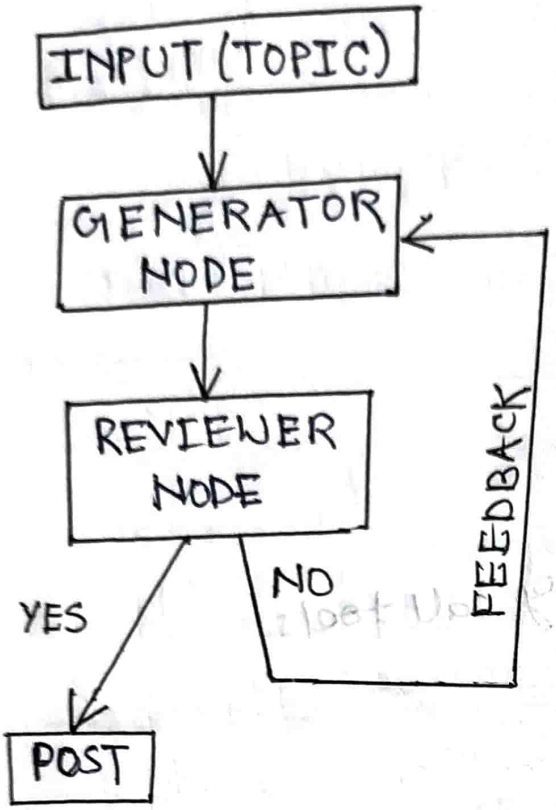
In [6]:

```
#USER PROGRAMME SELECTION
print("Welcome to the College Assistant\n")
print("1. BCA")
print("2. BBA")
print("3. B.com(hons)")
choice = input("\nEnter 1, 2 or 3: ")
programme_map = {
    "1": "BCA"
    "2": "BBA"
    "3": "B.com(hons)" }
student_programme = programme_map.get(choice,
    "BCA")
print(f"\n Great! You're set as {student_programme}
    Student.")

# CHAT LOOP
while True:
    user_query = input("You: ")
    if user_query.lower() in ["exit", "quit"]:
        break
    result = app.invoke({"programme": student_programme",
    "messages": [("human", user_query)] })
    print(f"Assistant: {result['messages'][-1].content}"
    )
```

→ Iterative Workflow

Here workflow uses cycles / backward edges to repeatedly execute nodes until a stopping conditions is reached.



↳ Key Points

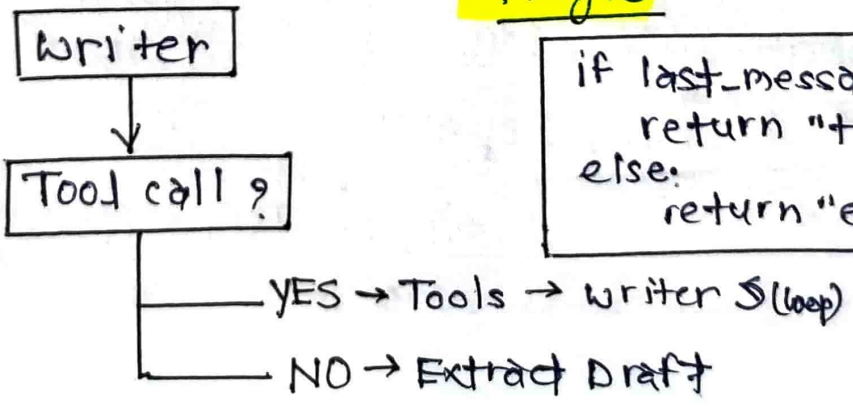
- It uses Back-ward edges to return to previous nodes.
- Repeats tasks until a stopping condn is met.
- It use ful for refinement, retry & Quality Improvement.
- It can conclude LLM + tool + feed back
- Uses router functions to control the loop
- recursive_limit prevents infinite loops.
- It can have multiple loops in the same graph.

↳ Router Functions

- should_use_tool() - controls the inner tool loop.

Logic

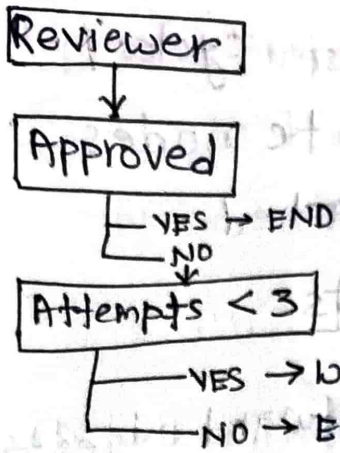
```
if last_message.tool_calls:
    return "tools"
else:
    return "extract_draft"
```



- **should_stop_looping()** : controls the outer quality loop

*stopping condns-

- is_approved == True
- attempt >= 3



↳ Recursive Limit

- safety limit for graph execution
- Prevents an infinite cycle.
- especially useful when LLM repeatedly call tools.

```
config = { "recursive_limit": 25 }
```

PROJECT-4
AUTONOMOUS LINKED-IN POST GENERATOR

→ Overview

This project automatically creates, reviews & improves LinkeIn post using AI, web search & iterative workflow.

- Take a "topic" from the user
- "writer llm" creates the LinkeIn post
- Uses Tavily web search when fresh / current information is needed.
- writer_node = creates/rewrites post & tracks attempts
- tool_node = Executes Tavily web search.
- extract_draft_node = Extracts & saves the draft
- reviewer_node = check quality & approves / rejects the post.
- "reviewer LLM" evaluates the post against quality criteria.
- If approved → workflow ends
- If rejects → feedback is sent back to the writer
- Maxm 3 attempts prevent excessive looping.
- Uses "conditional routing" to control both tool-calling & refinement loops.
- Uses "recursive_limit" as an additional safety mechanism when configured.

• Core Flow :

Topic → Writer → Tools → Draft → Reviewer → Approved ?

↓

END / writer

→ **1. IMPORTS**

In[1]:

```
import os
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
from langgraph.graph import StateGraph, START, END
from langgraph.prebuild import ToolNode
from langchain_groq import ChatGroq
from langchain_openai import ChatOpenAI
from langchain_tavily import TavilySearch
from dotenv import load_dotenv
load_dotenv()
```

→ **2. TOOL SETUP**

In[2]

```
search_tool = TavilySearch(max_results=3)
#creates web search tool
tools = [search_tool] # stores available tools
```

3. LLM SETUP

In[3]:

```
writer_llm = ChatOpenAI(model="gpt-4o-mini",
        temperature=0.7)
#This model used for writing & Temperature higher
  mean high creavity & low temperature low creativity
writer_llm_with_tools = writer_llm.bind_tools(tools)
# It allows writer to call tools
reviewer_llm = ChatGroq(model="llama-3.3-70b-versatile",
        temperature=0.2)
#This Model used for reviewing & give consistent judgement
```

4. STATE DEFINITION

In[4]:

```
class State(TypedDict):
    topic: str  #user's requested topic
    messages: Annotated[list, add_messages]
              #conversation & tool history
    draft: str  #draft for current LinkeIn post
    review_feedback: str  #Reviewer feedback
    is_approved: bool  #Approval status in bool (True/False)
    attempt: int  # No. of writing attempts
```

5. WRITER NODE

In[5]:

```
WRITER_SYSTEM_PROMPT = ("you are an expert LinkedIn
        content writer..")
#Instruction for writer, write more good prompt
```

```
def writer_node(state:State) -> dict:
    attempt = state.get("attempt", 0) + 1
    # Increase attempt count
    topic = state["topic"] # get user's topic
    previous_feedback = state["review_feedback"]
    #for get previous review

    if attempt == 1:
        user_message = (f"write a LinkedIn post on this topic{topic}"
            f"if you need current info search the web first")
        # creates the first draft
    else:
        user_message = (f"your previous draft on '{topic}' was
            rejected"
            f"Here is the reviewer's feedback: \n\n{previous_feedback}
              \n\n"
            f"write a new, improved draft that fixes every issue
              mentioned.")
        #Improved previous draft using feedback
    messages = [("system", WRITER_SYSTEM_PROMPT),
        ("human", user_message)]
    response = writer_llm_with_tools.invoke(messages)
    # It generate draft
    return {"messages": [("human", user_message), response],
        "attempt": attempt }
    # It save messages & save attempt no.
```

6. TOOL NODE & EXTRACT DRAFT NODE

In [ ]

```
tool_node = ToolNode(tools) # Automatically executes tool requests
def extract_draft_node(state:State) -> dict
    last_message = state["messages"][-1]
    print(f"\n Generated Post:\n{draft}")
    return {"draft": draft}
#Gets the latest AI message → extract its content → saves it as "draft"
                                                    in the state
```

7. REVIEWER NODE

In[7]:

```
REVIEWER_SYSTEM_PROMPT = "you are a strict LinkeIn
    content reviewer")#write detailed good prompt
def reviewer_node(state: State) -> dict:
    draft = state["draft"] #Get current draft
    prompt = (f"Review this LinkedIn post draft: \n"
      f"{draft} \n"
      f"Give your review")
    response = reviewer_llm.invoke(
       [("system", REVIEWER_SYSTEM_PROMPT),
        ("human", prompt)] )
    review_text = response.content.strip()
    is_approved = "APPROVED" in review_text.upper().split
                  ("FEEDBACK")[0]
    if "FEEDBACK:" in review_text:
       feedback = review_text.split("FEEDBACK:", 1)[1].
                  strip() #Extract feedback
    else:
       feedback = review_text # use complete review as
                                feedback
    return {"review_feedback": feedback,
            "is_approved": is_approved}
    #save feedback & save approval status
```

8. TOOL ROUTER

In[8]:

```
def should_use_tool(state: State):
    last_message = state["messages"][-1]
    #Get latest message
    if getattr(last_message, "tool_calls", None):
       return "tools" #If LLM requested a tool
    return "extract_draft"
    #otherwise extract the draft
```

9. LOOP ROUTER

In[9]:

```
def should_stop_looping(state:State):
    if state["is_approved"]:
        return END  #stop when approved
    if state["attempt"]>= 3:
        return END #stop after 3 attempts
    return "writer"
```

10. NODES, EDGES & COMPILE

In[10]:

```
#NODE
graph=StateGraph(state) #Create Langgraph workflow
graph.add_node("writer",writer_node)
graph.add_node("tools",toolnode)
graph.add_node("extract_draft",extract_draft_node)
graph.add_node("reviewer", reviewer_node)
#EDGES
graph.add_edge(START,"writer")
graph.add_conditional_edges("writer",should_use_tool)
#Decide: tool or draft extraction
graph.add_edge("tools","reviewer")
graph.add_edge("extract_draft","reviewer")
graph.add_conditional_edges("reviewer",should_stop_looping)
#COMPILE
app=graph.compile()
```

→ 11. USER INPUT & INITIAL STATE

In[1]:

```
#USER INPUT
topic = input("\n What topic do you want a LinkedIn
      post about? \n>").strip()
#Get topic from user

#INITIAL STATE
intial_state ={
    "topic": topic,
    "messages": [],
    "draft": "",
    "review_feedback": "",
    "is_approved": False,
    "attempt": 0, }
```

→ 12. RUN WORKFLOW & FINAL OUTPUT

In[2]:

```
#RUN WORKFLOW
final_state = app.invoke(intial_state)

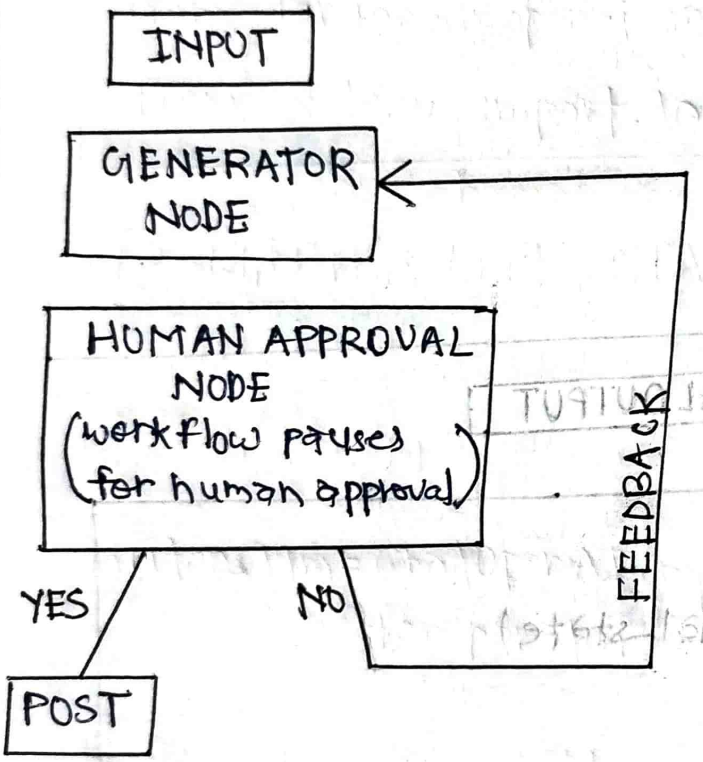
#FINAL OUTPUT
print(final_state["draft"])
print(f"Total attempts: {final_state['attempt']}")
print(f"Approved: {final_state['is_approved']}")
```

→ HITL (Human in the loop) Workflow

here human manually reviews / intervenes in an AI workflow before it continues.

↳ Key Points

- AI generates a result
- workflow pauses for human review.
- Human gives approval or feedback
- workflow resumes & either ends or improves the result.



↳ Main Components

- **interrupt()** = Pauses the workflow & waits for human input.
- **command(resume=...)** sends human input & resumes the workflow.
- **MemorySaver** : saves the workflow state while paused
- **thread_id** : Identifies a specific workflow/session.

PROJECT :5
HITL LINKED-IN POST GENERATOR

→ **overview**

AI generates a LinkedIn post → human reviews it → provides approval/feedback → AI imporves it iteratively until approved or max attempts reached.

- **User Input** - Takes a LinkedIn post topic.
- **Writer LLM** - Generates the post using GPT-4o-mini
- **Human review** - "interrupt()" pauses the workflow for manual review.
- **Human Decision** - Approve the post or provide feedback.
- **Feedback Loop** - Rejected post goes back to the writer for improvement.
- **Max 3 Attempts** - Prevents endless rewriting
- **MemorySaver** - saves workflow state during interruption.
- **Thread ID** - Identifies the current workflow session
- **Resume** - "command(resume=...)" continues the workflow after human input.

↳ **Flow**

Topic → Writer → Human Review → Approve
↓
Writer ← END / Feedback

1. IMPORTS & SETUP

In[1]:

```
import os
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
```

2. WRITER LLM & STATE DEFINITION

In[2]:

```
#WRITER LLM
writer_llm = ChatOpenAI(model="gpt-4o-mini",
                        temperature=0.7)

#STATE DEFINITION
class State(TypedDict):
    topic : str
    messages : Annotated[list, add_messages]
    draft : str
    review_feedback : str
    is_approved : bool
    attempt : int
```

3. WRITER PROMPT & WRITER NODE

In [3]:

```
WRITER_SYSTEM_PROMPT= ("Write Instruction Prompt
        for writer NODE")
def writer_node(state:State) -> dict:
    """Creates or rewrites the LinkedIn post"""
    attempt = state.get("attempt", 0) + 1
    #Increase attempt number
    topic = state["topic"]
    previous_feedback = state.get("review_feedback", "")
    #Get previous human feedback
    print(f"\n[Attempt {attempt}] Writer is
     drafting the post...")
    #Show current attempt
    if attempt == 1:
        user_message = f"Write a LinkedIn post on this
         topic: {topic}"
    #first attempt → create a new post
    else:
        user_message = (
            f"Your previous draft on '{topic}' was rejected.\n\n"
            f"Reviewer feedback: \n{previous_feedback}\n\n"
            f"Write a NEW improved LinkedIn post that fixes
              every issue mentioned.")
    #Later attempts → Improve using feedback
    messages = [("system", WRITER_SYSTEM_PROMPT),
                ("human", user_message)]
    response = writer_llm.invoke(messages)
    draft = response.content
    print(f"\n[Draft ready]\n{'-'*55}\n{draft}\n
        {'-'*55}")
    return {"draft": draft, "attempt": attempt}
```

4. HUMAN REVIEW NODE

In[4]:

```
def human_review_node(state:State) -> dict:
    """Pauses workflow & waits for human feedback"""
    print(f"\n[Reached human review- Attempt {
        state['attempt']}]")
    human_response= interrupt({
        "draft": state["draft"] #send draft to human
        "attempt":state["attempt"],
        #send attempt number
        "instruction": ("Type 'approved' to accept,
         or type your feedback"
        "to request a rewrite.") })
    response =human_response.strip()
    #Remove unnecessary spaces
    if response.lower() in ["approved", "approve" , "yes",
                            "ok", "good"]:
        return {
            "is_approved": True, #Mark as approved
            "review_feedback": "Approved by human"
            #store approval
        else:
            return{"is_approved":False,
                   "review_feedback":response}
    #Human rejected → store feedback
```

5. LOOP/ROUTING LOGIC

```
def should_stop_looping(state:State):
    if state['is_approved']:
        print("\n[Post approved by human. Ending workflow]")
        return END
    # If human approved → finish workflow
```

```
    if state['attempt']>=3:
        print("\n[Reached max 3 attempts. Ending
              with last draft.]")
        return END
    #stop after max^m 3 attempts
    print(f"\n[Rejected. Looping back to writer"
          f" for attempt {state['attempt']+1}...]"
         )
    return "writer"
```

6. NODE, EDGES, CHECKPOINT & COMPILE

In[6]:

```
graph = StateGraph(State)
graph.add_node("writer", writer_node)
graph.add_node("human_review", human_review_node)

graph.add_edge(START, "writer")
graph.add_edge("writer", "human_review")
graph.add_conditional_edge("human_review",
        should_stop_looping,
        {"writer": "writer", END: END, },)
#Human Review → writer or END

checkpoint = MemorySaver()
#create memory checkpoint system
app = graph.compile(checkpointer=checkpointer)
#compile graph with checkpointing
```

7. USER INTERFACE

In [9]:

```
print ("=" *55)
print ("Welcome to the LinkedIn Post Generater (HITL
        Edition)")
print("=" *55)
#Display application header

print("\nThis tool will draft a LinKed In Post for you,
       show it to")
print ("you for review & rewrite based on your feedback")
print("=" *55)
#Explain workflow to user

topic= input("\nwhat topic do you want a LinkedIn
       Post about? \n>").strip()
#Take topic from user

#INPUT VALIDATION

if not topic:
    print("\n No topic given. Exiting.")
# check whether user entered a topic
```

8. INITIAL STATE & SESSION

In [8]:

```
else:
    print("\nstarting generation... \n")
    config={"configurable": {
            "thread_id": "linkedIn_session_1"}}
    #Unique ID for this workflow session
```

```
initial_state={
    "topic": topic,
    "messages": [],
    "draft": "",
    "review_feedback": "",
    "is_approved": False
    "attempt": 0,}
#Intial state values
result= app.invoke(intial_state, config=config)
#start workflow
```

9.HUMAN-IN-THE-LOOP RESUME LOOP

In [9]:

```
While "__interrupt__" in result:
#continue while workflow is waiting for human input
    interrupt_data = result["__interrupt__"][0].
                                    value
    #Get information sent by interrupt()
    print("\n"+"="*55)
    print(f"DRAFT FOR YOUR REVIEW"
          f" (Attempt {interrupt_data['attempt']})")
    print("="*55)
    print(f"\n{interrupt_data['instruction']}")
    #show review instruction
    human_input = input("\nYour response:").strip()
    # Get human approval/feedback
    result = app.invoke(Command(resume=human_input),
                        config=config)
```

10. FINAL OUTPUT

In [10]:

```
print("\n" + "=" *55)
print("FINAL LINEDIN POST")
print("=" *55)
print(result["draft"])
print("=" *55)
#Display final LinkedIn Post

print(f"Total attempts:{result['attempt']}")
#Display total attempts
print(f"Approved by human:{result['is_approved']}")
#Display approval status
```